Лекция 4

Архитектура информационных систем

Данные, приложения, интеграции, технологии и безопасность



Результаты изучения темы

Нужно научиться выбирать устройство ИС под задачу предприятия и объяснять последствия выбора, включая отказ, повтор сообщения и ограничение доступа.

Описывать

Данные, приложения и способы обмена.

Сравнивать

Архитектурные стили и варианты размещения.

Проверять

Согласованность, качество и безопасность решения.

От информации к работающей системе

Начнём со смысла данных, затем рассмотрим распределение функций, обмен и техническую среду. Безопасность и качество проходят через все эти решения.

Данные
Что хранится и означает запись.

→

Приложения
Кто выполняет обработку.

→

Обмен
Как части взаимодей-ствуют.

→

Среда
Где и в каких условиях всё работает.

Архитектура данных

Архитектура данных определяет значимые информационные объекты, отношения, источники, ответственность и правила жизненного цикла сведений.

Смысл

Что обозначает объект и его атрибуты.

Организация

Как объекты связаны и идентифицируются.

Управление

Как сведения создают, проверяют, изменяют и используют.

Физическая схема таблиц — только часть архитектуры данных.

Учебный пример

Одинаковое поле не гарантирует одинаковый смысл

Две системы могут использовать поле «количество», но учитывать разные события или единицы. Интеграция начинается с согласования семантики.

Система А

Количество фактически выданного корма.

Система Б

Количество по запланированной заявке.

Риск

Суммы похожи по названию, но относятся к разным состояниям.

Учебный пример

Сущность, экземпляр и атрибут

Сущность представляет класс объектов; экземпляр — конкретный объект этого класса; атрибут — свойство, необходимое для его описания.

Сущность

Складская операция.

Экземпляр

Выдача № OP-017.

Атрибуты

Время, корм, площадка, количество и статус.

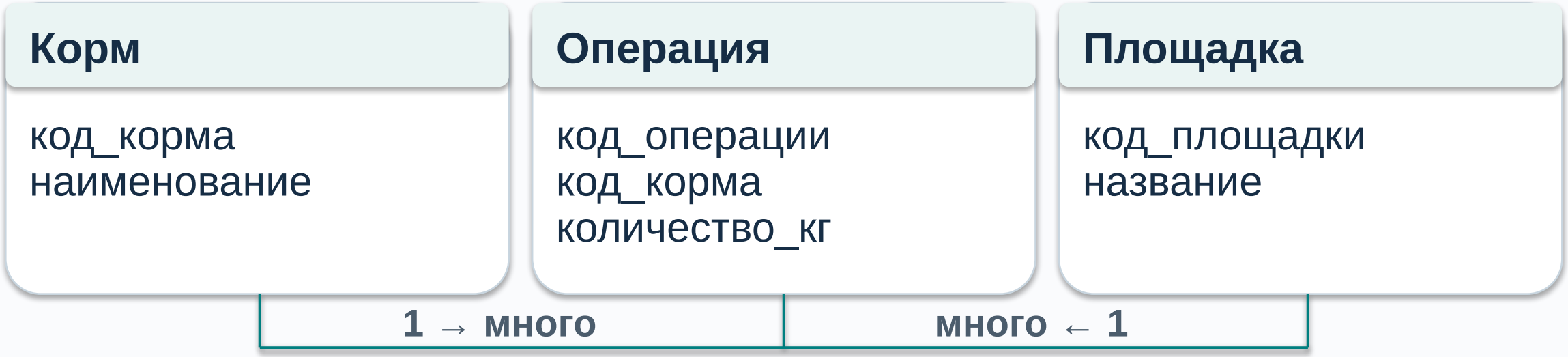
Три уровня модели данных

Уровни различаются степенью детализации и связью с реализацией. При переходе между ними должен сохраняться смысл предметной области.

Концептуальный	Основные сущности и отношения.
Логический	Атрибуты, ключи, ограничения и структура.
Физический	Таблицы, типы, индексы и способы хранения.

Концептуальная связь объектов

В учебной модели операция относится к определённому корму и площадке. Один корм и одна площадка могут участвовать во многих операциях.



Кратность следует из правил деятельности, а не из удобства рисунка.

Логическая модель уточняет ограничения

На логическом уровне устанавливают, какие сведения обязательны, как различать экземпляры и какие отношения допустимы.

Идентичность

У каждой операции есть устойчивый код.

Обязательность

Подтверждённая выдача связана с кормом и площадкой.

Ограничение

Количество выражено в согласованной единице.

Физическая модель реализует логическую

В конкретном хранилище выбираются типы данных, таблицы, индексы и другие механизмы. Техническое решение должно сохранять бизнес-ограничения.

Тип

Как представлены дата, количество и идентификатор.

Индекс

Как ускоряется нужный поиск.

Ограничение целостности

Как предотвращается недопустимая запись.

Индекс ускоряет определённые обращения, но не исправляет неверный смысл данных.

Ключ отделяет идентичность от названия

Ключ позволяет определить конкретный экземпляр. Отображаемое название может изменяться; это не должно автоматически создавать новый объект.

Идентификатор

FEED-001 сохраняет связь с одним объектом.

Наименование

Может быть исправлено или уточнено.

Внешняя ссылка

Операция хранит связь с объектом справочника.

Отношения и кратность

Отношение показывает связь экземпляров, а кратность — допустимое число связанных объектов. Важно различать обязательную и необязательную связь.

Одна ко многим

Одна поставка содержит несколько строк.

Обязательность

Строка должна относиться к поставке.

Проверка смысла

Правило подтверждается устройством деятельности.

Основные классы данных

Разделение помогает установить разные правила изменения и контроля.

Класс	Содержание	Пример
Справочные	Допустимые значения	Единицы измерения
Основные	Устойчивые объекты бизнеса	Корма, площадки
Операционные	События деятельности	Поступление, выдача

Авторитетный источник: System of Record

System of Record — система, признанная авторитетным источником определённых сведений. Это правило устанавливается для объекта или его атрибутов, а не обязательно для всех данных предприятия.

Назначить область

Например, сведения о подтверждённых выдачах.

→

Определить источник

Где запись получает принятый статус.

→

Установить обмен

Как другие системы получают и обновляют копии.

Владение данными

Владение означает содержательную ответственность: правила определения, качества, изменения и использования данных. Техническое хранение само по себе эту ответственность не определяет.

Владелец

Согласует смысл и правила.

Исполнитель

Создаёт или исправляет запись.

Техническая служба

Обеспечивает хранение и доступ.

Копия данных и неконтролируемое дублирование

Копии могут быть нужны для аналитики, производительности или автономной работы. Проблема возникает, когда неизвестны их происхождение, актуальность и правила согласования.

Управляемая копия

Есть источник, версия и способ обновления.

Опасное дублирование

Два независимых значения претендуют на роль истинного.

Цель архитектуры — управлять согласованностью, а не запретить любые копии.

Контроль качества данных

Качество проверяют относительно задачи и правил предметной области. Проверка формата необходима, но не подтверждает истинность хозяйственного события.

Формат

Количество является допустимым числом.

Связность

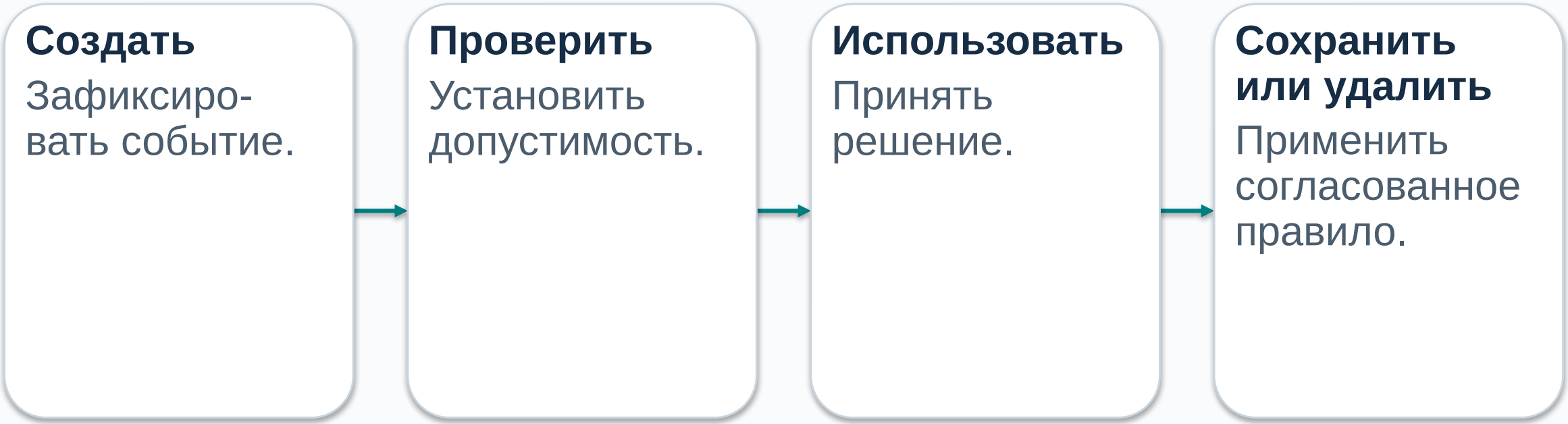
Код корма существует в справочнике.

Смысл

Количество и статус соответствуют реальной операции.

Жизненный цикл данных

Архитектура определяет не только создание записи, но и её проверку, использование, изменение, хранение и завершение жизненного цикла.



Происхождение и прослеживаемость данных

Прослеживаемость позволяет понять, из каких источников и преобразований получен показатель. Она особенно важна при расхождении сводного результата с первичными сведениями.

Источник

Конкретные операции выдачи.

→

Преобразование

Отбор подтверждённых записей и суммирование.

→

Показатель

Расход корма за выбранный день.

Из итогового числа должен быть понятен путь к его основаниям.

Операционные и аналитические данные

Операционная система поддерживает текущую деятельность; аналитическая — изучение и обобщение её результатов.

Вопрос	Операционная обработка	Аналитическая обработка
Основная задача	Записать и подтвердить	Сравнить и обобщить
Пример	Одна выдача	Расход по площадкам
Особое требование	Корректность операции	Согласованность показателей

Хранилище и витрина данных

Аналитическое хранилище объединяет данные для согласованного анализа. Витрина выделяет информацию под определённую область или группу вопросов.

Источники

Операционные системы и другие разрешённые сведения.

→

Подготовка

Сопоставление, проверка и преобразование.

→

Использование

Показатели, отчёты и аналитические представления.

Единицы, версии и время — часть смысла

Даже технически успешный обмен может создать неверный результат, если не согласованы единицы измерения, версии справочников и правила времени.

Единица

Килограммы не смешиваются с тоннами.

Версия

Известно, по какому справочнику проверяли запись.

Время

Различаются время события и время регистрации.

Архитектура приложений

Архитектура приложений описывает состав систем, распределение функций, взаимодействие и зависимости. Она связывает бизнес-требования с программной поддержкой.

Состав

Какие приложения используются.

Обязанности

За что отвечает каждое приложение.

Связи

Какие данные и сервисы оно предоставляет другим.

Оценка портфеля приложений

Бизнес-ценность и техническое состояние — разные критерии. Важную, но проблемную систему нельзя автоматически вывести из эксплуатации.

	Техническое состояние: плохое	Техническое состояние: хорошее
Высокая ценность	**Обновить**	**Развивать**
Низкая ценность	**Вывести / объединить**	**Переоценить назначение**

Решение учитывает стоимость перехода и зависимости, а не только место в матрице.

Учебный пример

Функциональное покрытие приложениями

Матрица показывает, какие функции поддерживаются и где возникает дублирование.

Функция	Складская ИС	Учётная ИС	Отчётный модуль
Регистрация выдачи	Основная	Повторный ввод	Нет
Сверка	Частично	Основная	Нет
Суточная сводка	Исходные записи	Учётные данные	Основная

Учебный пример

Приложение, компонент и сервис

Приложение объединяет функции для деятельности. Компонент — выделенная часть реализации. Сервис предоставляет определённую возможность через согласованный интерфейс.

Приложение

Система учёта движения кормов.

Компонент

Модуль проверки операций.

Сервис

Проверка и сохранение переданной выдачи.

Клиент и сервер — роли взаимодействия

Клиент инициирует обращение, сервер предоставляет запрошенную возможность. Один компонент может быть сервером для пользователя и клиентом другой системы.

Клиент

Отправляет запрос и использует ответ.

Сервер

Обрабатывает обращение по установленным правилам.

Взаимодействие

Контракт определяет допустимый запрос и результат.

Три функциональных слоя приложения

Укрупнённо различают представление, бизнес-логику и работу с данными. Такое разделение описывает обязанности, а не обязательно три компьютера.

Представление	Взаимодействует с пользователем.
Бизнес-логика	Применяет правила предметной области.
Доступ к данным	Читает и изменяет сохраняемые сведения.

Логический слой и физическое звено

Слой разделяет обязанности в программе. Звено размещения показывает, где выполняется обработка. Эти оси архитектуры нужно различать.

Логические слои

Интерфейс, правила и доступ к данным.

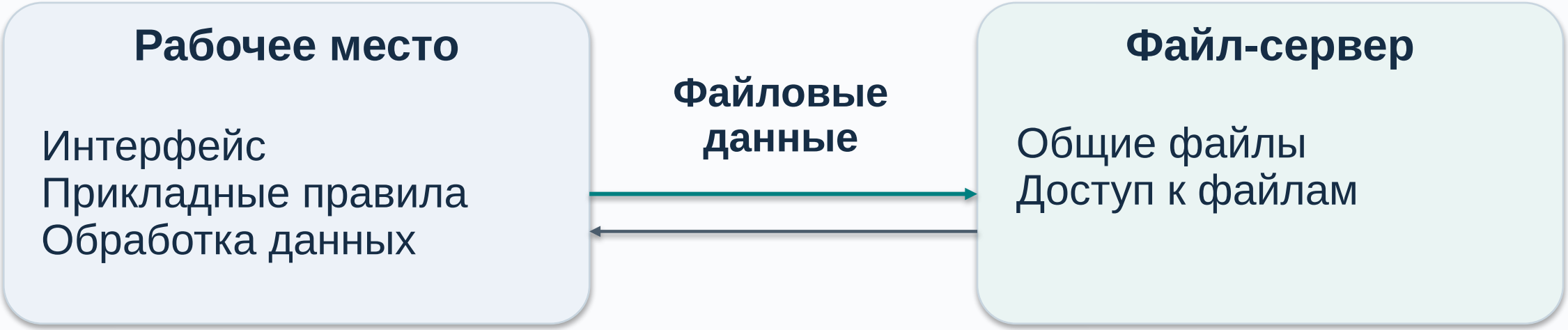
Физические звенья

Клиентское устройство, сервер приложения и сервер БД.

Трёхслойное приложение может работать на одном узле; трёхзвенное — размещено раздельно.

Файл-серверная архитектура

В классической файл-серверной схеме общие файлы хранятся на сервере, а значительная часть прикладной обработки выполняется на клиенте.



Разделяемое хранение не решает автоматически согласованность одновременных изменений.

Границы применения файл-серверной схемы

Такая схема может быть простой для небольшой задачи, но требует оценки сетевого обмена, блокировок, целостности и восстановления файлов.

Условия

Небольшое число участников и понятный характер операций.

Риски

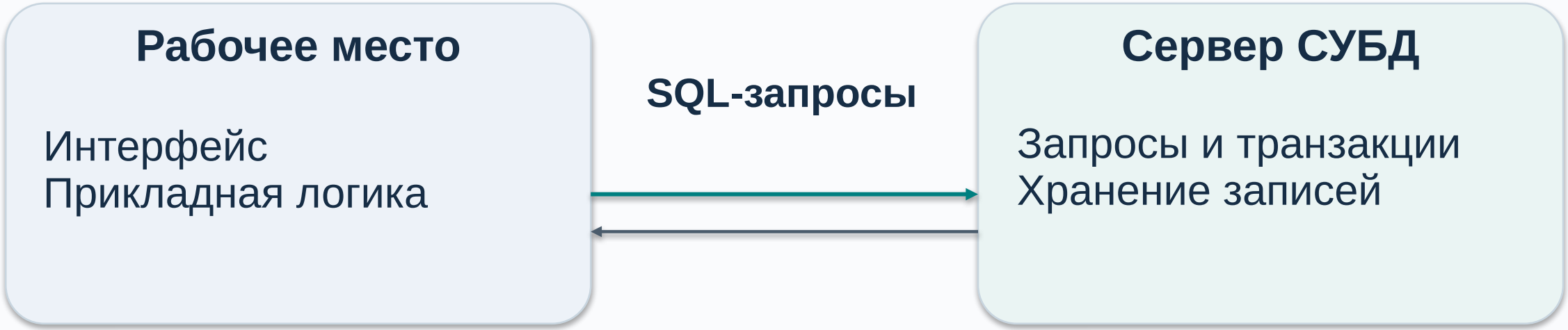
Конкурирующие изменения и передача больших объёмов.

Проверка

Что происходит при обрыве сети и одновременной записи.

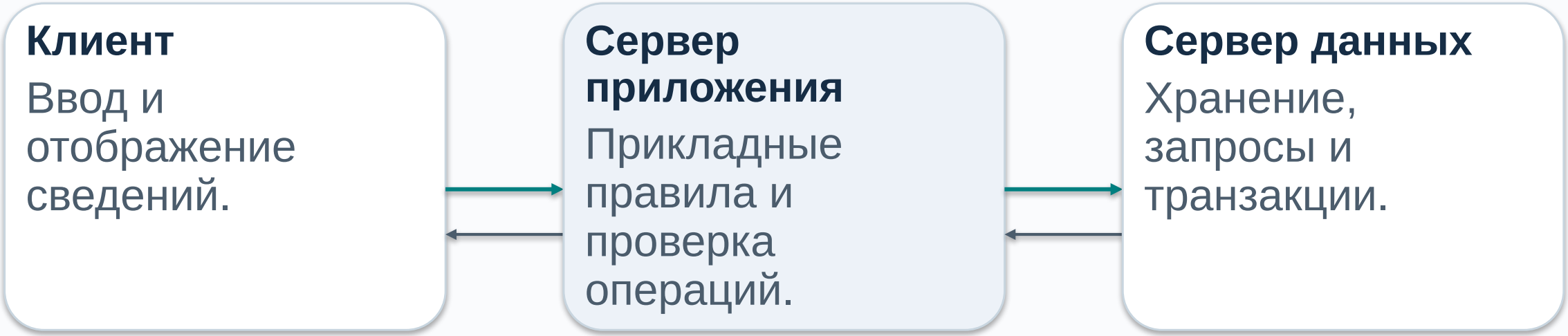
Двухзвенная клиент-серверная схема

Клиентское приложение обращается к серверной СУБД через запросы. Часть бизнес-логики может находиться на клиенте, часть — в серверных механизмах.



Трёхзвенная архитектура

Между клиентом и хранилищем выделяется сервер приложения. Он централизует прикладные правила и взаимодействие с данными.



Дополнительное звено даёт новые возможности, но усложняет эксплуатацию.

Монолитное приложение

Монолит объединяет основные программные части в единую единицу сборки и развёртывания. Внутри него могут существовать хорошо разделённые логические слои.

Плюс

Простое совместное развёртывание и локальные вызовы.

Риск

Изменение одной части может требовать выпуска всего приложения.

Важное условие

Внутренняя модульность определяет качество, а не само слово «монолит».

Учебный пример

Модульный монолит

Модульный монолит сохраняет одну единицу развёртывания, но разделяет обязанности на модули с явными границами и интерфейсами.

Одно приложение — одна единица развёртывания

Операции
Ввод и подтверждение.

→

Справочники
Коды и единицы.

→

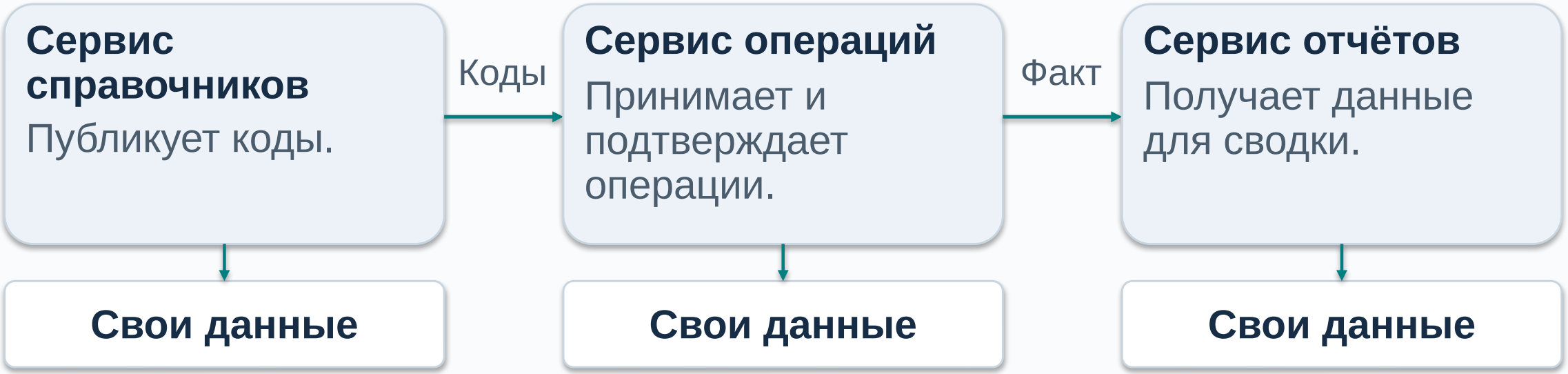
Отчёт
Сводка принятых записей.

Граница модуля должна ограничивать зависимости, а не только имя папки.

Учебный пример

Микросервисная архитектура

Микросервисный подход разделяет систему на сервисы с отдельными обязанностями и независимым жизненным циклом. Связь между ними требует явных сетевых контрактов.



Распределение функций имеет стоимость

Переход от локального вызова к сетевому взаимодействию создаёт задержки, возможность частичного отказа, повторной доставки и несовместимости версий.

Сеть

Ответ может опоздать или потеряться.

Состояние

Части системы могут видеть разные версии.

Эксплуатация

Нужно наблюдать несколько взаимодействующих компонентов.

Больше сервисов не означает автоматически лучшую архитектуру.

Сравнение монолита и микросервисов

Выбор делается по требованиям и возможностям команды, а не по возрасту технологии.

Критерий	Модульный монолит	Микросервисы
Развёртывание	Одна единица	Несколько единиц
Взаимодействие	Чаще локальное	Сетевые контракты
Эксплуатация	Меньше распределённых связей	Больше наблюдения и координации

Когда простое решение предпочтительнее

Для небольшого процесса с одной командой модульный монолит может быть достаточным. Независимое масштабирование и выпуск отдельных частей должны иметь реальное основание.

Сначала спросить

Какая проблема требует разделения?

→

Оценить последствия

Какие отказы и компетенции добавятся?

→

Выбрать

Наименее сложный вариант, выполняющий требования.

Сервисно-ориентированная архитектура

SOA — Service-Oriented Architecture — организует взаимодействие через сервисы с определёнными возможностями и контрактами. Принцип не привязан к одному продукту или протоколу.

Сервис

Предоставляет полезную возможность.

Контракт

Определяет обращение и результат.

Согласование

Позволяет объединять разные приложения.

Интеграционная шина — один из вариантов

ESB — Enterprise Service Bus — может обеспечивать маршрутизацию, преобразование и согласование сообщений. Но SOA не требует обязательного наличия одной центральной шины.

Возможность

Общие механизмы обмена.

Риск

Центральная зависимость и сложное сопровождение.

Альтернатива

Другие варианты маршрутизации и прямые контракты.

Контракт взаимодействия

Контракт задаёт, какие сообщения допустимы, что они означают и как стороны реагируют на успех, ошибку, повтор и изменение версии.

Содержание

Поля, типы, единицы и смысл.

Поведение

Предусловия, результат и ошибки.

Совместимость

Версия и правила изменения.

API: интерфейс программного взаимодействия

API — Application Programming Interface — описывает доступные программные операции и правила обращения к ним. API бывает локальным или сетевым.

Операция

Например, создать запись выдачи.

Вход

Обязательные сведения и полномочия.

Выход

Результат выполнения или объяснимая ошибка.

Наличие API не гарантирует согласованность смысла данных.

Синхронное взаимодействие «запрос — ответ»

Отправитель ожидает ответ на обращение. Нужно определить допустимое время ожидания и смысл ответа: запрос принят или операция уже завершена.



Ответ должен иметь однозначный бизнес-смысл.

Асинхронное взаимодействие

При асинхронном обмене отправка и завершение обработки разделены во времени. Отправитель может продолжить работу, но должен иметь способ узнать состояние операции.

Отправить
Передать сообщение.

→

Обработать
Выполнить действие позже.

→

Установить результат
Получить событие, статус или иной согласованный ответ.

Асинхронность не означает отсутствие ответственности за незавершённую операцию.

Команда и событие

Команда выражает просьбу выполнить действие. Событие сообщает о уже произошедшем факте. Их смешение делает поведение интеграции неоднозначным.

Команда

«Подтвердить выдачу OP-017».

Событие

«Выдача OP-017 подтверждена».

Последствие

Событие не должно незаметно превращаться в право выполнить любое действие.

Пакетный обмен и ETL

Пакетный обмен передаёт сведения группами по расписанию или событию. ETL — Extract, Transform, Load — извлечение, преобразование и загрузка данных.

Извлечь

Получить согласованный набор из источника.

→

Преобразовать

Проверить и сопоставить формат и смысл.

→

Загрузить

Записать результат и зафиксировать статус.

Пакет должен иметь понятные границы, идентификатор и правило повторной обработки.

Топология обмена

Схема соединений влияет на количество зависимостей и место общих правил.

Вариант	Преимущество	Ограничение
Точка — точка	Простой отдельный обмен	Рост числа связей
Общий узел	Централизованные правила	Критичная зависимость
Сообщения / события	Разделение во времени	Контроль доставки и состояния

Минимальный состав сообщения об операции

Кроме бизнес-полей нужны сведения, позволяющие проследить обработку и отличить новый факт от повторной доставки.

Идентичность

Код операции и источник.

Содержание

Корм, площадка, количество и единица.

Состояние

Время события, версия и статус.

Версии и идентификаторы

Идентификатор связывает сообщения с одним объектом. Версия позволяет понять, какое состояние или формат представлен.

Один объект

OP-017 остаётся той же операцией.

Новая версия

Исправление количества имеет основание и номер версии.

Новый формат

Изменение контракта не должно ломать старого получателя молча.

Идемпотентность

Идемпотентная обработка позволяет повторить одно и то же обращение без дополнительного бизнес-эффекта. Это особенно важно при неизвестном результате первой отправки.

Первый запрос

Операция OP-017 учтена.

→

Повторный запрос

Система узнаёт тот же идентификатор.

→

Результат

Вторая выдача не создаётся.

Повторная доставка сообщения и повторение хозяйственного события — не одно и то же.

Тайм-аут не доказывает неисполнение

Тайм-аут означает, что ожидаемый ответ не получен вовремя. Сервер мог не принять запрос, продолжать обработку или завершить её с потерей ответа.

Неизвестность

Клиент не знает фактический результат.

Опасный ход

Создать новую операцию с новым кодом.

Безопасный подход

Уточнить статус или повторить обращение по согласованному правилу.

Повторы должны быть ограничены

Повторная попытка полезна для временной ошибки, но бесконечные повторы могут усиливать перегрузку и скрывать неисправность.

Условие

Повтор допустим для данного типа операции.

Режим

Ограничены число попыток и частота.

Завершение

После исчерпания попыток есть понятный маршрут разбора.

Ошибочное сообщение не должно исчезать

Если сообщение нельзя обработать автоматически, его сохраняют с причиной и передают на разбор. Иногда для этого используют отдельную очередь проблемных сообщений.

Зафиксировать
Исходное сообщение и причину ошибки.

→

Назначить
Ответственного за разбор.

→

Продолжить
Исправить и повторить либо завершить с объяснимым отказом.

Локальная транзакция

Транзакция объединяет связанные изменения данных в согласованную операцию. Гарантии действуют в пределах конкретного механизма, а не автоматически для всех внешних систем.

Внутри границы

Запись операции и изменение остатка согласуются.

При ошибке

Не допускается частично принятый результат по правилам транзакции.

Вне границы

Отправка во внешнюю систему требует отдельной организации.

Согласованность распределённых данных

В распределённой системе изменения могут становиться видимыми в разных компонентах не одновременно. Архитектура должна определить допустимое расхождение и способ его устранения.

Требование

Когда получатель обязан видеть актуальное состояние.

Механизм

Передача, подтверждение и повтор.

Контроль

Поиск и устранение несогласованных состояний.

Частичный отказ

Частичный отказ означает, что одни компоненты продолжают работать, а другие недоступны или отвечают неправильно. Пользовательский сценарий может при этом остаться незавершённым.

Работает

Площадка сохранила локальную выдачу.

Не работает

Связь с центральной системой недоступна.

Нужно решить

Как показать статус и позже согласовать результат.

Модель размещения

Модель размещения показывает, на каких узлах выполняются компоненты и какие технические связи нужны для их взаимодействия.

Узлы

Рабочее место, сервер, площадка или облачная среда.

Размещение

Какие компоненты находятся на каждом узле.

Каналы

Какие соединения необходимы для сценария.

Три группы инфраструктурных ресурсов

Вычисления, хранение и сеть взаимозависимы. Ускорение одной части может не изменить общий результат, если ограничение находится в другой.

Вычисления

Выполнение алгоритмов и сервисов.

Хранение

Сохранение и получение данных.

Сеть

Передача между участниками обработки.

Вертикальное и горизонтальное масштабирование

Вертикальное масштабирование увеличивает ресурсы узла; горизонтальное — число экземпляров или узлов. Возможность такого изменения зависит от архитектуры.

Вертикально

Больше памяти или вычислительной мощности.

Горизонтально

Несколько экземпляров обработки.

Ограничение

Общая БД или внешний сервис могут остаться узким местом.

Балансировка нагрузки

Балансировщик распределяет обращения между доступными экземплярами. Для корректной работы нужно определить состояние сеанса и совместимость обработки.

Распределение

Как выбирается экземпляр для запроса.

Проверка

Как определяется работоспособность экземпляра.

Состояние

Где хранятся данные, необходимые следующему запросу.

Единая точка отказа

Единая точка отказа — элемент, нарушение работы которого останавливает существенную функцию системы. Искать её нужно по пользовательскому сценарию.

Пример

Общая БД без проверенного восстановления.

Зависимость

Все регистрации требуют её доступности.

Архитектурный вопрос

Допустима ли остановка и как восстанавливается работа.

Резервная копия и восстановление

Наличие копии не подтверждает возможность восстановить нужный результат. Необходимо проверить данные, конфигурацию, доступы и выполнение сценария после восстановления.

Сохранить

Необходимые данные и параметры.

→

Восстановить

Получить работоспособную среду.

→

Проверить

Сверить состояния и выполнить значимые операции.

RPO и RTO: разные ограничения восстановления

RPO (Recovery Point Objective) задаёт допустимую потерю данных по времени; RTO (Recovery Time Objective) — целевой срок восстановления работы после нарушения.

RPO

Как далеко назад во времени допустимо потерять изменения.

RTO

За какое время требуется восстановить услугу.

Проверка

Резервирование и процедура восстановления должны поддерживать оба ограничения.

Понятие облачных вычислений

Облачная модель предоставляет по требованию доступ к общему пулу настраиваемых ресурсов, которые можно оперативно выделять и освобождать. Простое размещение сервера в ЦОД ещё не описывает эту модель.

Получение

Ресурсы предоставляются как услуга.

Гибкость

Объём ресурсов может изменяться.

Учёт

Использование измеряется и управляется.

IaaS, PaaS и SaaS

Модели различаются уровнем предоставляемой возможности и оставшейся ответственностью пользователя.

Модель	Предоставляется	Пользователь работает с
IaaS	Инфраструктурой	ОС и приложениями
PaaS	Средой исполнения	Своим приложением
SaaS	Готовым приложением	Настройками и данными

Модели развёртывания облака

Модели развёртывания различаются кругом пользователей и организацией доступа. Само название модели не устанавливает качество и безопасность.

Публичное

Инфраструктура доступна широкому кругу потребителей.

Частное

Исключительное использование одной организацией.

Общественное

Используется сообществом с общими требованиями.

Гибридное

Соединяет различимые облачные инфраструктуры.

Ответственность в облаке не исчезает

Провайдер и пользователь отвечают за разные части сервиса. Пользователь сохраняет обязанности по данным, полномочиям и корректной настройке своей области.

Провайдер

Предоставляет согласованный уровень сервиса.

Потребитель

Управляет своими пользователями, данными и настройками.

Совместная граница

Определяется договором, моделью услуги и механизмами контроля.

Виртуальная машина и контейнер

Оба подхода помогают изолировать и размещать программную среду, но делают это на разных уровнях. Контейнер не равен отдельному физическому серверу.

Виртуальная машина

Имеет гостевую ОС на виртуализированных ресурсах.

Контейнер

Изолирует среду приложения, обычно разделяя ядро ОС.

Архитектурный выбор

Учитывает совместимость, безопасность и сопровождение.

Распределённая система

Распределённая система объединяет взаимодействующие вычислительные компоненты на разных узлах. Для пользователя они могут предоставлять общую функцию, но сохраняют сетевые зависимости.

Размещение

Обработка выполняется не в одном процессе или узле.

Взаимодействие

Компоненты передают сообщения.

Следствие

Нужно учитывать задержки, состояние и частичные отказы.

Локальная обработка и работа без сети

Часть действий можно выполнять рядом с источником данных или на устройстве пользователя. Это полезно при нестабильной связи, но требует последующего согласования.

Локально

Зафиксировать событие и проверить доступные правила.

→

При восстановлении связи

Передать накопленные изменения.

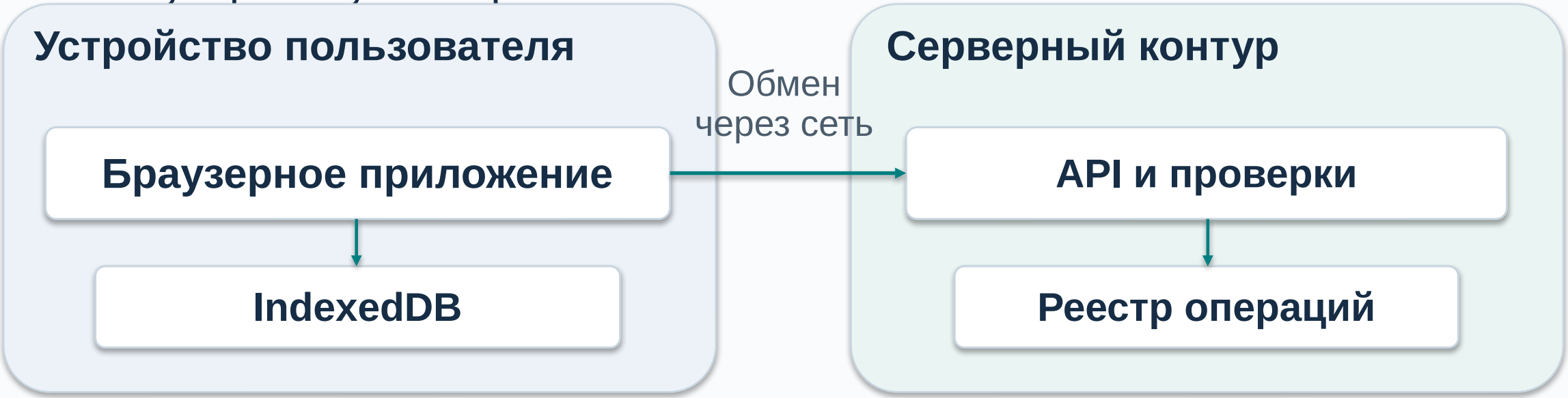
→

Централизованно

Проверить полномочия, версии и итоговую согласованность.

IndexedDB: данные внутри браузера

IndexedDB — браузерный интерфейс хранения структурированных данных с ключами, индексами и транзакциями. Он поддерживает локальную работу веб-приложения.



Браузерное хранилище само по себе не синхронизирует данные разных пользователей.

IndexedDB, SQLite и MySQL — разные задачи

Сравнивать следует область применения, а не «современность» названия.

Средство	Типичный контекст	Способ работы
IndexedDB	Браузер пользователя	Объекты и ключи
SQLite	Встраиваемая локальная БД	SQL в процессе приложения
MySQL	Серверная реляционная БД	Клиент-серверный SQL

Локальное хранение не заменяет автоматически общий многопользовательский реестр.

Локальные данные: синхронизация и риск потери

Данные браузера зависят от ограничений хранения и действий пользователя. Архитектура должна учитывать очистку, квоты, конфликт версий и потерю устройства.

Синхронизация

Как изменения достигают общего реестра.

Конфликт

Как сравниваются две версии одного объекта.

Сохранность

Что делать при потере локальной копии.

Безопасность — свойство согласованной архитектуры

Безопасность рассматривает защищаемые данные и действия, участников, границы доверия и сценарии нарушения. Перечень продуктов защиты не заменяет эту модель.

Что защищаем

Данные, услуги и значимые операции.

От чего

Подмена, раскрытие, изменение, отказ.

Как действуем

Предотвращение, обнаружение и восстановление.

Активы и границы доверия

Актив — значимый для системы объект защиты. Граница доверия показывает, где меняются условия проверки отправителя, данных или полномочий.

Актив

Подтверждённая запись выдачи.

Граница

Переход от браузерного ввода к принятой серверной записи.

Проверка

Кто отправил данные и какие действия ему разрешены.

Аутентификация и авторизация

Аутентификация устанавливает, кто обращается. Авторизация определяет, имеет ли этот субъект право выполнить конкретное действие с конкретным объектом.

Кто вы?

Проверка идентичности пользователя.

Что вам разрешено?

Проверка полномочий на действие.

Важное различие

Вошедший пользователь не получает право на любые записи.

Минимально необходимые полномочия

Права должны соответствовать задачам роли и области её ответственности. Более широкий доступ требует отдельного основания и контроля.

Действие

Создание не равно подтверждению.

Область

Своя площадка не равна всем площадкам.

Время

Временное исключение должно иметь срок завершения.

Пример разграничения прав

Проверка прав выполняется в доверенном контуре, а не только скрытием кнопки интерфейса.

Роль	Разрешено	Ограничение
Кладовщик	Создать выдачу	В своей области
Подтверждающий	Принять факт	По назначенной площадке
Аналитик	Читать сводку	Без изменения операций

Шифрование и содержательная корректность

Защита передачи и хранения помогает обеспечить конфиденциальность и целостность в заданных условиях. Она не доказывает правильность хозяйственных сведений.

Канал защищён

Сообщение передано по предусмотренному защищённому соединению.

Смысл проверяется отдельно

Код площадки, количество и полномочия могут быть недопустимыми.

Нельзя принять неверный документ только потому, что он пришёл по защищённому каналу.

Журнал значимых действий

Журнал позволяет восстановить, кто, когда и на каком основании изменил объект. Сам журнал тоже требует управления доступом и правилами хранения.

Субъект

Кто выполнил действие.

Изменение

Что было изменено и каков результат.

Основание

Какая операция, версия или причина связана с записью.

Не следует помещать пароли, секреты и лишние персональные сведения в журналы.

Учебный пример

Сценарий угрозы и мера защиты

Угроза описывается через конкретное действие и последствие. Мера должна объяснять, что она предотвращает, обнаруживает или помогает восстановить.

Сценарий

Пользователь передал чужой код площадки.

→

Последствие

Чужие операции могли попасть в учёт.

→

Мера

Проверить соответствие площадки полномочиям отправителя.

ИИ-компонент в архитектуре ИС

ИИ-компонент рассматривается как часть системы с входами, выходами, ограничениями и проверкой. Он не должен незаметно получать полномочия владельца процесса.

Назначение

Например, предложить соответствие наименований.

Проверка

Установить допустимость и качество предложения.

Ответственность

Определить, кто подтверждает значимый результат.

Данные для ИИ и пределы передачи

Перед использованием внешнего или внутреннего ИИ-сервиса нужно определить допустимые данные, полномочия и правила использования результата.

Минимизация

Передавать только необходимую информацию.

Разрешённость

Проверить ограничения доступа и обработки.

Контроль

Не принимать правдоподобный ответ за подтверждённый факт.

Учебный пример

Резервный сценарий при ошибке ИИ

Если ИИ не дал пригодного результата, система должна сохранить управляемость. Значимое действие может быть остановлено или передано человеку.

Неуверенный результат

Соответствие корма не установлено.

→

Остановка действия

Запись не подтверждается автоматически.

→

Разбор

Ответственный выбирает допустимый объект и фиксирует основание.

Качественные атрибуты архитектуры

Архитектурное качество выражается в способности выполнять требования при значимых условиях. Общие слова нужно переводить в проверяемые сценарии.

Работа

Доступность и производительность.

Изменение

Модифицируемость и сопровождаемость.

Контроль

Безопасность и наблюдаемость.

Учебный пример

Сценарий качества

Сценарий связывает воздействие, условия, затронутую часть, ожидаемую реакцию и измеримый результат.

Воздействие

После потери ответа повторяется отправка.

→

Реакция

Система узнаёт уже принятую операцию.

→

Критерий

В реестре и отчёте остаётся один факт расхода.

Компромисс: доступность и согласование

Автономный ввод помогает продолжать работу без сети, но не гарантирует мгновенную согласованность со всеми участниками. Нужно определить допустимые состояния.

Преимущество

Работа на площадке не останавливается.

Цена

Появляется очередь несинхронизированных изменений.

Контроль

Пользователь видит локальный и центральный статусы отдельно.

Учебный пример

Кейс: ограничения исходного состояния

В «Учебном хозяйстве» используются складской ввод, отдельный учёт и сводный отчёт. Обмен частично ручной, связь с площадкой нестабильна.

Данные

Названия и идентификаторы не согласованы.

Обмен

Возможны повторы и задержки.

Работа

Нужно сохранить запись и понятный статус операции.

Учебный пример

Кейс: выбираем модули, а не лишние сервисы

Для учебного объёма выбираем приложение с явными модулями операций, справочников и отчёта. Независимое развёртывание каждого модуля пока не требуется.

Одно приложение — одна единица развёртывания

Операции
Ввод и подтверждение.

→

Справочники
Коды и единицы.

→

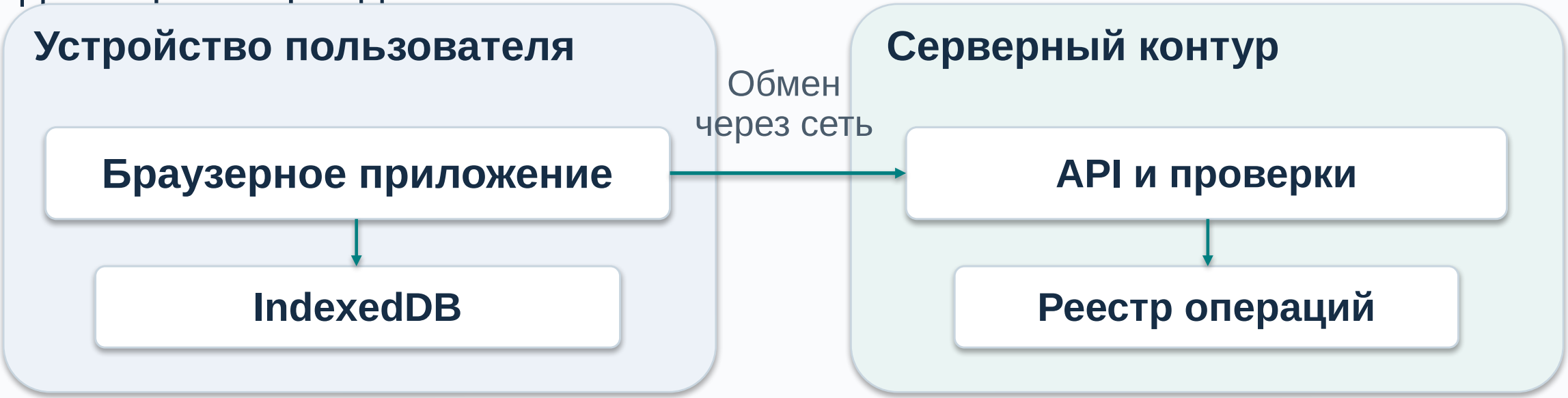
Отчёт
Сводка принятых записей.

Выбран модульный вариант; микросервисы не являются обязательной целью курса.

Учебный пример

Кейс: локальная запись и общий реестр

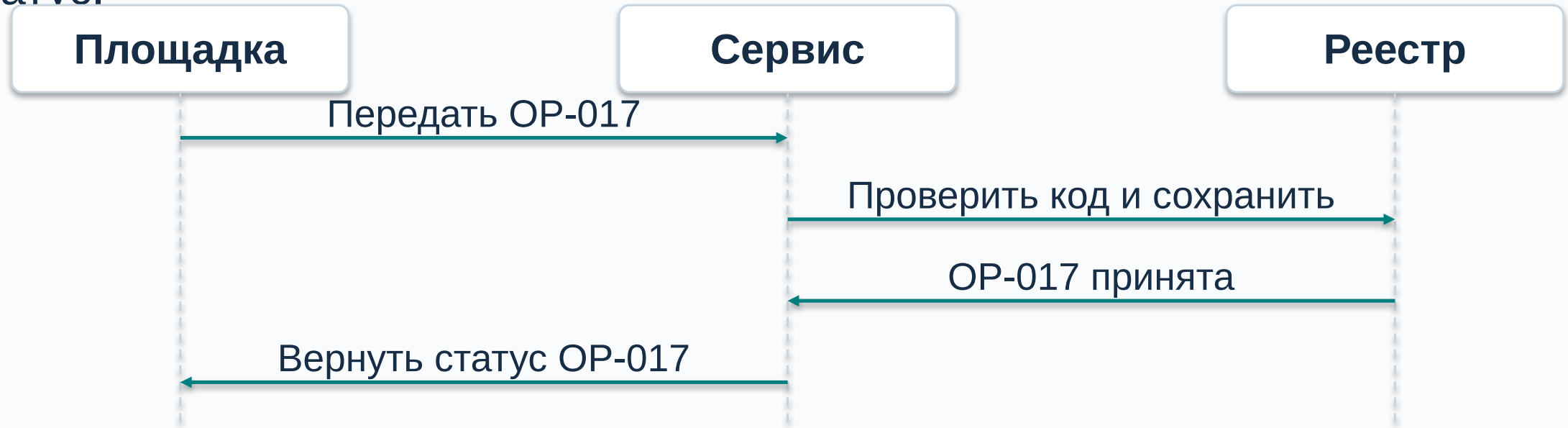
При потере связи браузер сохраняет неподтверждённые локальные операции. Сервер после получения проверяет полномочия, идентификатор и данные.



Локальная запись не объявляется принятой центральной системой без подтверждения.

Кейс: возвращение связи

После восстановления сети отправляется та же операция с тем же идентификатором. Только полученный результат меняет центральный статус.



Исходная операция сохраняется до установленного подтверждения.

Учебный пример

Кейс: повтор не создаёт второй расход

Если ответ потерялся, площадка повторяет передачу OP-017. Центральный сервис проверяет идентичность и возвращает согласованный результат.

На входе

Тот же код и тот же смысл операции.

→

В реестре

Обнаружена уже принятая запись.

→

На выходе

Подтверждение без второго учётного эффекта.

Учебный пример

Кейс: проверка безопасности обмена

Недостаточно проверить, что поле «площадка» заполнено. Нужно установить, вправе ли отправитель создавать операцию для указанной площадки.

Аутентификация

Известен субъект обращения.

Авторизация

Площадка входит в область полномочий.

Аудит

Результат и причина отказа доступны ответственному.

Учебный пример

Кейс: почему выбранная архитектура достаточна

Выбор объясняется задачей: достоверный отчёт, управляемая автономность и ограниченная сложность. Границы проверки остаются явными.

Поддержано

Идентичность операций и явные статусы.

Упрощено

Одна единица серверного развёртывания.

Не доказано

Промышленная нагрузка и работа на всех реальных площадках.

Перенос: приём заявлений без стабильной сети

Сотрудник принимает заявления на выезде. Что требуется кроме локального браузерного хранилища?

Идентичность

Устойчивый код каждого заявления.

Согласование

Повторная отправка и центральное подтверждение.

Безопасность

Правила доступа, хранения и обработки данных.

Проверка понимания

Ответьте коротко, опираясь на архитектурные последствия.

1. Чем логический слой отличается от звена размещения?
2. Почему тайм-аут не доказывает, что операция не выполнена?
3. Что должна обеспечить идемпотентная обработка?
4. Почему IndexedDB не заменяет автоматически серверную БД?
5. Чем аутентификация отличается от авторизации?

Ответы к проверке

Ключевые различия:

1. Слой разделяет обязанности; звено — физическое размещение.
2. Ответ мог потеряться после успешного выполнения.
3. Повтор обращения не создаёт дополнительного бизнес-эффекта.
4. Локальное хранилище не обеспечивает общую синхронизацию.
5. Аутентификация — кто; авторизация — что разрешено.

От устройства ИС к выбору целевого состояния

Архитектурное решение должно объединить смысл данных, обязанности приложений, обмен, размещение и правила безопасности.

Не перечень технологий

Каждый компонент связан с требованием.

Не только нормальный случай

Показаны ошибки и частичные отказы.

Не обещание

Свойства подтверждаются подходящими проверками.

Следующая лекция: целевая архитектура, разрывы, переход и оценка решения.

Источники и материалы для повторения

Учебные материалы курса и первичные источники. Полные ссылки и привязка к теме находятся в заметках соответствующих слайдов.

[2] Пособие, 2026

[9] Сурин: материалы курса

[17] Пособие, гл. 4

[18] OMG UML 2.5.1

[19] Архитектура ИС: текст

[20] Молдованова, 2015

[21] NIST SP 800-145

[22] W3C IndexedDB

[23] MDN Storage

[24] SQLite

[25] MySQL

[26] OWASP

[27] Пособие, гл. 9–11

[28] SEI ATAM

[29] NIST SP 800-34